DIVVY TRIPS 2020 Q1

SQL Query-Based Rider Behaviour Analysis

Casual Riders vs. Annual Members

Prepared by: **Sarthak Sehgal**

Executive Summary

This project compares casual riders and annual members using trip-level bike-share data. The SQL analysis focuses on ride volume, ride duration, day-of-week activity, and weekday-versus-weekend patterns. The business goal is to identify behaviour-based opportunities that the marketing team can test to encourage casual riders to purchase annual memberships.

The source capstone reports 426,887 raw trip records and an analysis-ready dataset of 426,387 records after 500 rides with negative, zero, or greater-than-24-hour durations were excluded. Reported results show that casual riders take longer trips and use the service more evenly across weekdays and weekends, while members account for most rides and are more weekday-oriented.

Project element	Details
Dataset	Divvy Trips 2020 Q1
Raw data	426,887 rows across 13 fields
Prepared data	426,387 rides according to the source report
SQL dialect	Google BigQuery Standard SQL
Core analyses	Ride volume, trip duration, day of week, weekday/weekend

Source accuracy note: the original capstone describes Python, Pandas, and Matplotlib as the tools used. This revised report adds BigQuery SQL equivalents that show how to reproduce the reported checks and metrics. The figures below are carried over from the source report and were not recalculated by executing SQL in this document.

Business Objective and Questions

Business question: How do casual riders and annual members differ in riding behaviour, and which patterns could inform membership-conversion campaigns?

• How many trips are taken by members and casual riders, and what share of rides does each group represent?

• How do average and median trip durations compare by rider type?

• Which days have the highest ride counts for each group?

• How does the weekday/weekend mix differ between members and casual riders?

Key stakeholders are the marketing team, executive/management team, and marketing leadership, who can use the findings to develop and evaluate segmented membership campaigns.

Dataset Overview

Field	Purpose in analysis
ride_id	Trip identifier used for counting and uniqueness checks.
started_at / ended_at	Trip timestamps used to calculate duration.
member_casual	Primary segment: member or casual rider.
rideable_type	Bike type recorded in the trip dataset.
Station fields and coordinates	Available for optional station and location analysis.

Data Quality and Raw-Data Profiling

The source report records 426,887 rows, 13 columns, no duplicate rows, and four missing values in destination-related fields. It reports no missing values in ride_id, started_at, ended_at, or member_casual.

SQL query: count records, check identifiers, and profile missing values

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT ride_id) AS distinct_ride_ids,
  COUNTIF(ride_id IS NULL) AS missing_ride_id,
  COUNTIF(started_at IS NULL) AS missing_started_at,
  COUNTIF(ended_at IS NULL) AS missing_ended_at,
  COUNTIF(member_casual IS NULL) AS missing_rider_type,
  COUNTIF(end_station_name IS NULL) AS missing_end_station_name,
  COUNTIF(end_station_id IS NULL) AS missing_end_station_id,
  COUNTIF(end_lat IS NULL) AS missing_end_lat,
  COUNTIF(end_lng IS NULL) AS missing_end_lng
FROM `your_project.raw.divvy_trips_2020_q1`;
```

How this produces the result: COUNT(*) returns the raw row count; COUNT(DISTINCT ride_id) checks identifier uniqueness; COUNTIF(column IS NULL) counts missing values in each field.

Reference result from the source report: 426,887 rows, no duplicate rows, and four missing destination/location values. Replace the example table path before running the query.

SQL query: inspect customer categories

```
SELECT member_casual, COUNT(*) AS total_rides
FROM `your_project.raw.divvy_trips_2020_q1`
GROUP BY member_casual
ORDER BY total_rides DESC;
```

The source report's raw-data check records two customer categories: 378,407 member records and 48,480 casual records before cleaning.

SQL query: identify repeated ride IDs

```
SELECT ride_id, COUNT(*) AS record_count
FROM `your_project.raw.divvy_trips_2020_q1`
GROUP BY ride_id
HAVING COUNT(*) > 1;
```

This query returns any ride_id appearing more than once. The source report states that no duplicate rows were found.

Ride-Duration Validation and SQL Data Preparation

The source capstone says started_at and ended_at were initially stored as text/object values. The queries below safely convert them to timestamps, calculate ride duration in minutes, and create calendar fields for the weekday/weekend analysis.

SQL query: classify invalid duration records

```
WITH parsed AS (
  SELECT
    SAFE_CAST(started_at AS TIMESTAMP) AS started_at_ts,
    SAFE_CAST(ended_at AS TIMESTAMP) AS ended_at_ts
  FROM `your_project.raw.divvy_trips_2020_q1`
),
durations AS (
  SELECT TIMESTAMP_DIFF(ended_at_ts, started_at_ts, SECOND) AS duration_seconds
  FROM parsed
)
SELECT
  COUNTIF(duration_seconds < 0) AS negative_duration_records,
  COUNTIF(duration_seconds = 0) AS zero_duration_records,
  COUNTIF(duration_seconds > 86400) AS over_24_hour_records,
  COUNTIF(duration_seconds IS NULL) AS missing_or_unparseable_timestamps
FROM durations;
```

Reference result: the source report identifies 117 negative-duration rides, 93 zero-duration rides, and 290 rides longer than 24 hours. These categories total 500 exclusions.

SQL query: create the analysis-ready table

```
CREATE OR REPLACE TABLE `your_project.analytics.divvy_trips_clean` AS
WITH parsed AS (
  SELECT
    ride_id, rideable_type,
    SAFE_CAST(started_at AS TIMESTAMP) AS started_at,
    SAFE_CAST(ended_at AS TIMESTAMP) AS ended_at,
    start_station_name, start_station_id,
    end_station_name, end_station_id,
    start_lat, start_lng, end_lat, end_lng,
    LOWER(TRIM(member_casual)) AS member_casual
  FROM `your_project.raw.divvy_trips_2020_q1`
),
derived AS (
  SELECT *,
    TIMESTAMP_DIFF(ended_at, started_at, SECOND) / 60.0
      AS ride_length_minutes,
    CASE EXTRACT(DAYOFWEEK FROM started_at)
      WHEN 1 THEN 'Sunday' WHEN 2 THEN 'Monday'
      WHEN 3 THEN 'Tuesday' WHEN 4 THEN 'Wednesday'
      WHEN 5 THEN 'Thursday' WHEN 6 THEN 'Friday'
      WHEN 7 THEN 'Saturday'
    END AS day_name,
    CASE
      WHEN EXTRACT(DAYOFWEEK FROM started_at) IN (1, 7)
        THEN 'Weekend'
      ELSE 'Weekday'
    END AS day_group,
    CASE EXTRACT(DAYOFWEEK FROM started_at)
      WHEN 1 THEN 7
      ELSE EXTRACT(DAYOFWEEK FROM started_at) - 1
    END AS day_order
  FROM parsed
)
SELECT *
FROM derived
WHERE started_at IS NOT NULL
  AND ended_at IS NOT NULL
  AND member_casual IN ('member', 'casual')
  AND ride_length_minutes > 0
  AND ride_length_minutes <= 1440;
```

How this produces the result: SAFE_CAST parses timestamps, TIMESTAMP_DIFF calculates elapsed seconds (converted to minutes), CASE/EXTRACT derives calendar groups, and WHERE retains valid positive durations of up to 24 hours. The source report states that 426,387 rows remained.

Before execution, update the raw and analytics table paths and check that timestamp strings parse correctly in your BigQuery dataset.

Ride Volume by Rider Type

SQL query: total rides and share of all rides

```
SELECT
  member_casual,
  COUNT(*) AS total_rides,
  ROUND(
    100 * SAFE_DIVIDE(COUNT(*), SUM(COUNT(*)) OVER ()),
    2
  ) AS share_of_rides_pct
FROM `your_project.analytics.divvy_trips_clean`
GROUP BY member_casual
ORDER BY total_rides DESC;
```

How this produces the result: COUNT(*) counts rides per rider category. The windowed SUM(COUNT(*)) OVER () calculates the overall ride total, and SAFE_DIVIDE calculates each category's percentage share.

Rider category	Rides reported	Share of rides
Member	378,344	88.73%
Casual	48,043	11.27%
Total	426,387	100.00%

Interpretation: members account for most trips in this dataset; casual riders contribute about 11.3% of recorded rides.

Average and Median Ride Duration

SQL query: compare average and median duration

```
SELECT DISTINCT
  member_casual,
  ROUND(
    AVG(ride_length_minutes) OVER (PARTITION BY member_casual),
    2
  ) AS avg_ride_minutes,
  ROUND(
    PERCENTILE_CONT(ride_length_minutes, 0.5)
      OVER (PARTITION BY member_casual),
    2
  ) AS median_ride_minutes
FROM `your_project.analytics.divvy_trips_clean`
ORDER BY member_casual;
```

How this produces the result: AVG calculates the mean ride length; PERCENTILE_CONT at 0.5 calculates the median; PARTITION BY computes each metric separately for members and casual riders.

Rider category	Average ride	Median ride
Member	11.50 minutes	8.57 minutes
Casual	37.03 minutes	21.15 minutes

Interpretation: casual riders average approximately 3.2 times the ride duration of members. Membership messaging could be tested with casual riders who take longer or repeated rides.

Weekly Ride Patterns

SQL query: rides by day of week and rider category

```
SELECT
  day_order,
  day_name,
  member_casual,
  COUNT(*) AS total_rides
FROM `your_project.analytics.divvy_trips_clean`
GROUP BY day_order, day_name, member_casual
ORDER BY day_order, member_casual;
```

How this produces the result: the query groups every ride by its derived day_name and rider type, counts trips, and sorts weekdays in calendar order using day_order.

Day	Member rides	Casual rides
Monday	61,916	4,785
Tuesday	69,692	5,225
Wednesday	63,969	5,846
Thursday	61,236	4,819
Friday	55,477	5,103
Saturday	30,095	7,445
Sunday	35,959	14,820

Reported finding: members peak on Tuesday (69,692 rides) and are lowest on Saturday (30,095). Casual riders peak on Sunday (14,820), followed by Saturday (7,445).

Weekday vs. Weekend Behaviour

SQL query: ride counts and within-group percentages

```
WITH grouped AS (
  SELECT member_casual, day_group, COUNT(*) AS rides
  FROM `your_project.analytics.divvy_trips_clean`
  GROUP BY member_casual, day_group
)
SELECT
  member_casual,
  day_group,
  rides,
  ROUND(
    100 * SAFE_DIVIDE(
      rides, SUM(rides) OVER (PARTITION BY member_casual)
    ),
    2
  ) AS share_within_rider_type_pct
FROM grouped
ORDER BY member_casual, day_group;
```

How this produces the result: the CTE first counts rides by rider type and day group. A windowed sum then becomes each rider type's denominator so the weekday/weekend percentages add up to 100% within that rider category.

Rider category	Weekday	Weekend
Casual	25,778 (53.66%)	22,265 (46.34%)
Member	312,290 (82.54%)	66,054 (17.46%)

Interpretation: members are strongly weekday-oriented. Casual riders have a more balanced weekday/weekend mix. This supports relative distribution differences; it does not mean casual riders have more total weekend trips than members.

Average Ride Duration by Day

SQL query: average duration for each day and rider type

```
SELECT
  day_order,
  day_name,
  member_casual,
  ROUND(AVG(ride_length_minutes), 2) AS avg_ride_minutes
FROM `your_project.analytics.divvy_trips_clean`
GROUP BY day_order, day_name, member_casual
ORDER BY day_order, member_casual;
```

How this produces the result: AVG(ride_length_minutes) is calculated separately for each combination of day and rider type. GROUP BY defines those groups, and ORDER BY keeps the days in calendar order.

Day	Casual average	Member average
Monday	27.51 min	11.13 min
Tuesday	30.63 min	11.35 min
Wednesday	39.12 min	11.21 min
Thursday	31.46 min	11.18 min
Friday	35.72 min	11.08 min
Saturday	40.05 min	12.14 min
Sunday	42.28 min	13.61 min

Reported finding: casual rides last longer on every day. Casual average duration is highest on Sunday (42.28 minutes); member average duration also peaks Sunday (13.61 minutes).

Business Insights and Recommendations

Findings derived by the SQL summaries

• **Ride volume:** members make up 88.73% of trips; casual riders account for 11.27%.

• **Ride duration:** casual rides average 37.03 minutes, compared with 11.50 minutes for members.

• **Timing:** member ride volume peaks on Tuesday; casual rider volume peaks on Sunday.

• **Weekday/weekend mix:** weekend trips comprise 46.34% of casual rides and 17.46% of member rides.

Recommended business actions

• Test membership messaging that highlights annual-plan value for casual riders who take longer or repeated rides.

• Test selected weekend and Sunday campaigns, when casual rider activity is highest in this dataset.

• Use distinct campaigns for the weekday-oriented member segment and the more balanced casual segment.

• Measure campaign exposure, conversion, new memberships, and retention before scaling initiatives.

SQL Techniques Used

SQL technique	How it was used
COUNT / COUNT DISTINCT	Count trips and check ride ID uniqueness.
COUNTIF / IS NULL	Count missing values in selected fields.
SAFE_CAST	Parse timestamp fields safely.
TIMESTAMP_DIFF	Calculate ride duration.
CASE / EXTRACT	Create day names and weekday/weekend groups.

SQL technique	How it was used
GROUP BY / ORDER BY	Aggregate results by rider category and day.
AVG / PERCENTILE_CONT	Calculate average and median ride duration.
Window functions / SAFE_DIVIDE	Calculate percentage shares within groups.

Execution and Reproducibility

• Load the Q1 CSV into a BigQuery raw table with the original 13 columns.

• Replace `your_project.raw.divvy_trips_2020_q1` and `your_project.analytics.divvy_trips_clean` with your actual table paths.

• Run profiling and duplicate checks before creating the cleaned table.

• Execute the table-creation query, then run the analysis queries against the cleaned table.

• Compare query outputs with the reference figures in this report and investigate any discrepancies.

The queries are written in Google BigQuery Standard SQL. COUNTIF, SAFE_CAST, SAFE_DIVIDE, TIMESTAMP_DIFF, and PERCENTILE_CONT may need syntax changes for other database engines.

Integrity note: the uploaded capstone describes Python/Pandas/Matplotlib as its original tools. This report now documents SQL query equivalents for each reported calculation so the analysis can be reproduced in SQL; it does not claim that these SQL queries were executed against the dataset during the original analysis.

DETAILED BUSINESS RECOMMENDATIONS

Divvy Trips 2020 Q1 | SQL Query-Based Rider Behaviour Analysis

This section expands the brief recommendations in the main report into specific actions linked to the findings in the source capstone. Figures below are reported in the supplied capstone and should be verified by running the SQL queries on the dataset.

1. Target Casual Riders Taking Longer Trips

Evidence from the report: average trip duration is 37.03 minutes for casual riders versus 11.50 minutes for members. The reported median durations are 21.15 and 8.57 minutes, respectively.

Recommendation: test membership messaging that explains the value and convenience of an annual membership for casual riders whose usage is frequent or whose rides are long. Avoid assuming every long ride is a likely conversion; use the campaign as a measured pilot.

Success measures: membership conversion rate, cost per conversion, and repeat usage after conversion. These require campaign/conversion data in addition to trip records.

SQL evidence: compare average duration and longer-trip share

```
SELECT
  member_casual,
  COUNT(*) AS total_rides,
  ROUND(AVG(ride_length_minutes), 2) AS avg_ride_minutes,
  COUNTIF(ride_length_minutes >= 30) AS rides_30_minutes_or_more,
  ROUND(100 * SAFE_DIVIDE(
    COUNTIF(ride_length_minutes >= 30), COUNT(*)
  ), 2) AS rides_30_minutes_or_more_pct
FROM `your_project.analytics.divvy_trips_clean`
GROUP BY member_casual
ORDER BY avg_ride_minutes DESC;
```

The 30-minute cut-off is a proposed analytical segment, not a threshold specified by the source report. Review other thresholds before using it for targeting.

2. Align Campaigns with Casual-Rider Activity

Evidence from the report: Sunday is the highest-volume day for casual riders (14,820 rides). Casual rides are split 53.66% on weekdays and 46.34% on weekends, while member rides are 82.54% on weekdays and 17.46% on weekends.

Recommendation: pilot weekend-focused promotions, with particular attention to Sunday activity. Compare the pilot with a control group or a comparable non-campaign period rather than assuming timing alone causes conversion.

Success measures: campaign response, membership starts, cost per new member, and subsequent engagement. The source trip table does not contain campaign exposure or membership conversion records.

SQL evidence: compare trip volume and duration by rider type and day

```
SELECT
  member_casual,
  day_name,
  day_group,
  COUNT(*) AS total_rides,
  ROUND(AVG(ride_length_minutes), 2) AS avg_ride_minutes
FROM `your_project.analytics.divvy_trips_clean`
GROUP BY member_casual, day_name, day_group
ORDER BY
  CASE day_name
    WHEN 'Monday' THEN 1 WHEN 'Tuesday' THEN 2
    WHEN 'Wednesday' THEN 3 WHEN 'Thursday' THEN 4
    WHEN 'Friday' THEN 5 WHEN 'Saturday' THEN 6
    WHEN 'Sunday' THEN 7
  END,
  member_casual;
```

IMPLEMENTATION & MEASUREMENT PLAN

3. Use Different Messages for Different Rider Behaviours

Evidence from the report: members peak on Tuesday (69,692 rides) and have a weekday-oriented usage pattern. Casual riders peak on Sunday (14,820 rides) and have a more balanced weekday/weekend pattern. Casual riders also take longer rides on every day listed in the report.

Recommendation: avoid a one-size-fits-all message. Test separate creative and timing for casual riders, including longer-ride value messaging and weekend/Sunday relevance. Treat these as hypotheses to test, not proven causes of conversion.

SQL evidence: rank peak riding days for each rider category

```
WITH day_summary AS (
  SELECT
    member_casual,
    day_name,
    COUNT(*) AS total_rides,
    ROUND(AVG(ride_length_minutes), 2) AS avg_ride_minutes
  FROM `your_project.analytics.divvy_trips_clean`
  GROUP BY member_casual, day_name
)
SELECT
  *,
  DENSE_RANK() OVER (
    PARTITION BY member_casual
    ORDER BY total_rides DESC
  ) AS ride_volume_rank
FROM day_summary
ORDER BY member_casual, ride_volume_rank;
```

4. Test, Measure, and Scale What Works

• **Establish a baseline:** record existing conversion and membership-start rates before a campaign.

• **Run a controlled pilot:** where feasible, compare campaign-exposed audiences with a similar control group.

• **Evaluate results:** compare conversion rate, cost per new member, and post-conversion retention.

• **Scale cautiously:** expand only if results improve against the baseline and control, while monitoring whether gains persist.

Measurement limitation: the ride dataset contains trip-level fields and a ride_id, but does not show campaign impressions, a persistent customer identifier, or membership conversion events. Therefore, SQL over this dataset can describe riding behaviour, but campaign effectiveness must be evaluated using a separate, appropriately linked and privacy-safe marketing results dataset.

Summary Action Plan

Finding	Recommended action	How to evaluate
Casual average ride: 37.03 min vs. member: 11.50 min	Test annual-plan value messaging for long or repeat-trip segments.	Conversion rate; cost per conversion; repeat use.
Casual Sunday rides: 14,820; 46.34% of casual trips occur on weekends	Pilot weekend/Sunday-targeted campaign timing.	Pilot vs. control; new memberships; cost per member.
Members are weekday-oriented; casual activity is more balanced	Use separate messaging and timing for each audience.	Compare campaign response and conversion by segment.

Final Recommendation

Divvy should test a segmented membership-conversion strategy focused on casual riders with longer or repeat trips and on periods of high casual-rider activity, especially Sundays. Messaging and timing should differ from approaches aimed at weekday-oriented members. The organization should validate the strategy through a measured campaign pilot using conversion and retention data before scaling it.

SQL is used here to establish the behavioural segments and activity patterns. The campaign recommendation is a business hypothesis supported by the reported patterns, not a causal conclusion from trip records alone.